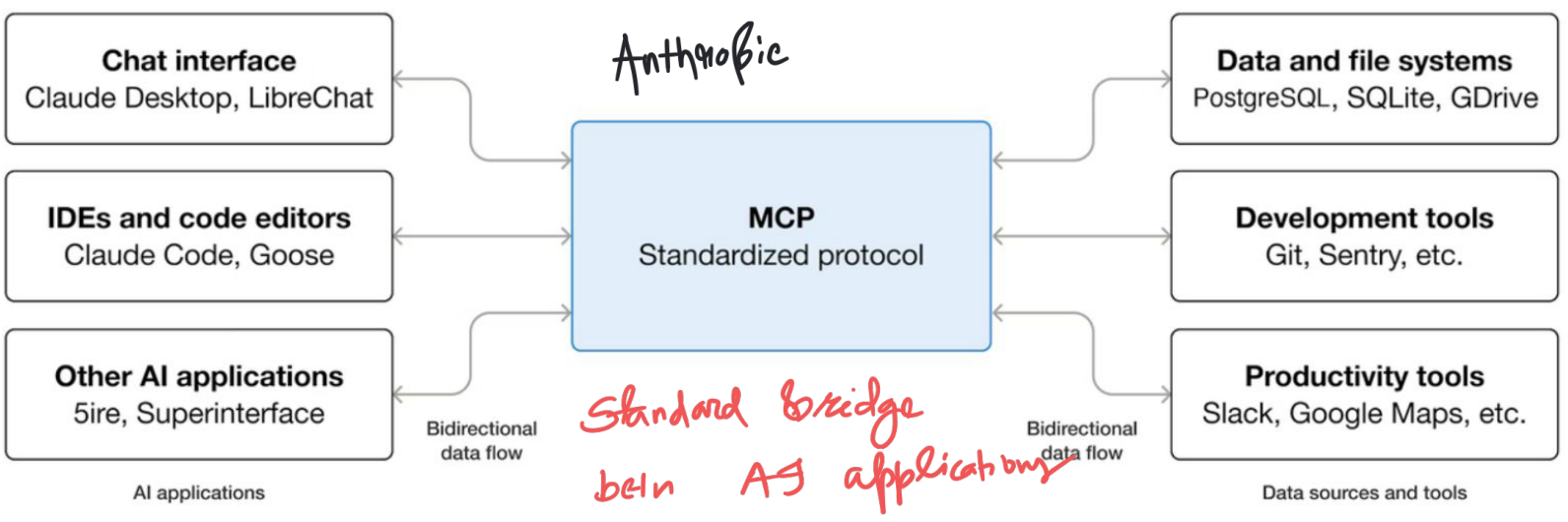
Model Context Protocol

Satya Pattnaik

Bluetooth
HDMI
USB-C
Aux

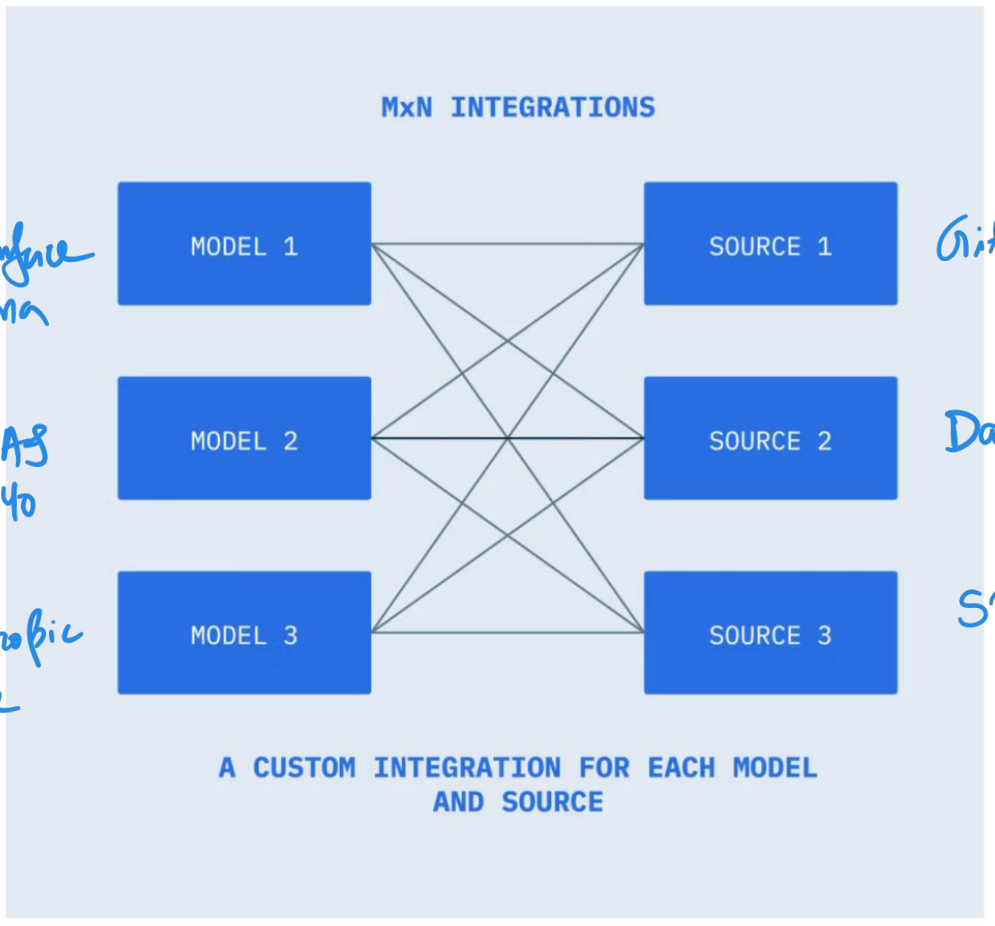
USB-C



Standard bridge
btwn AI applications

LLM Chat Agent ⟷ Tools

no. of connections = #of models X #of sources



Huggingface Llama

OpenAI GPT-4o

Anthropic Claude

Github

Database

S3

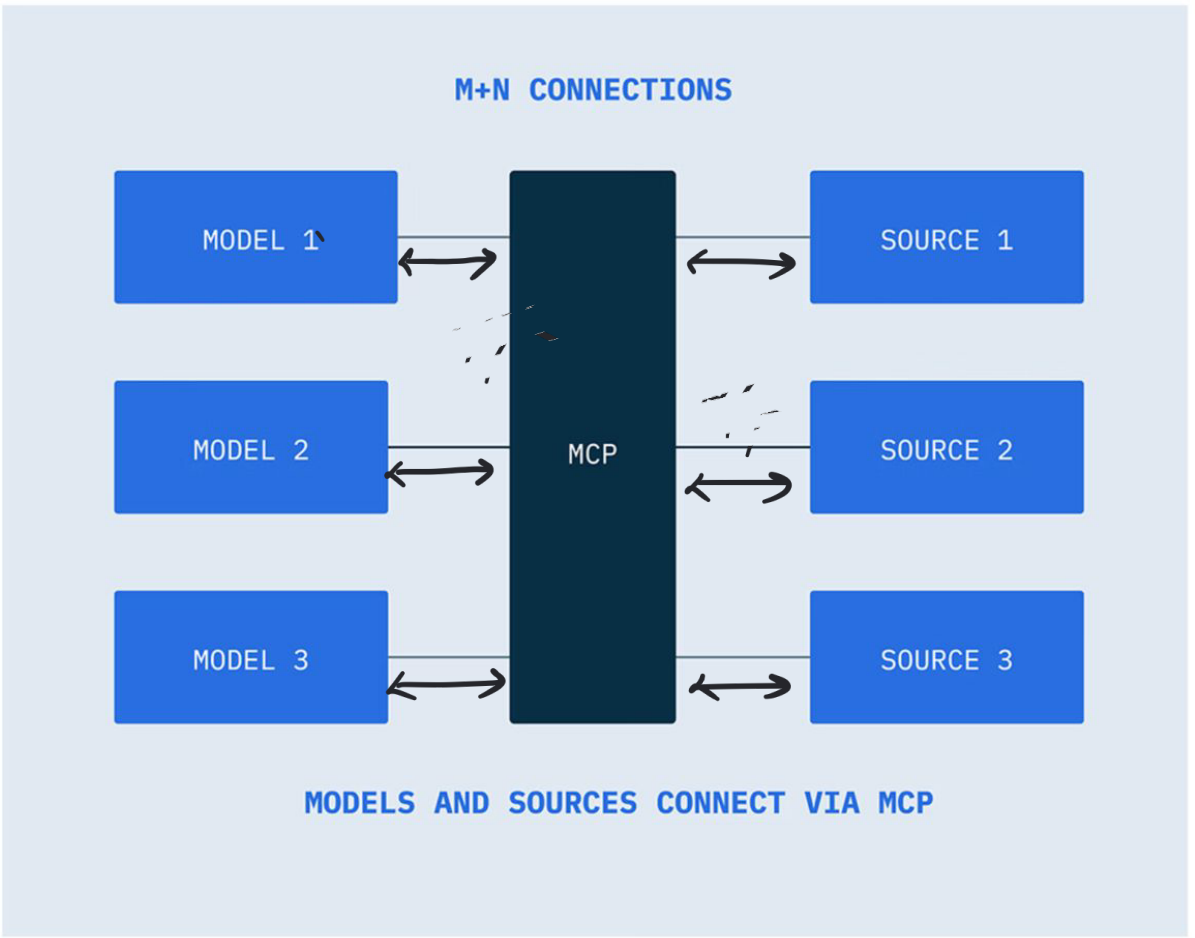
→ too many integrations

→ high maintenance costs

→ duplicate work

→ not a scalable approach

M+N integration



1] Server (Hosted on a server port)

Expose tools to AI Model

AWS Cloud Servic

2] Host (Connected directly to the user)

→ Claude Desktop → ChatGPT → Windsurf

→ Claude Web →Cursor

3] Client (abstracted inside Host)

Hidden component within the Host application

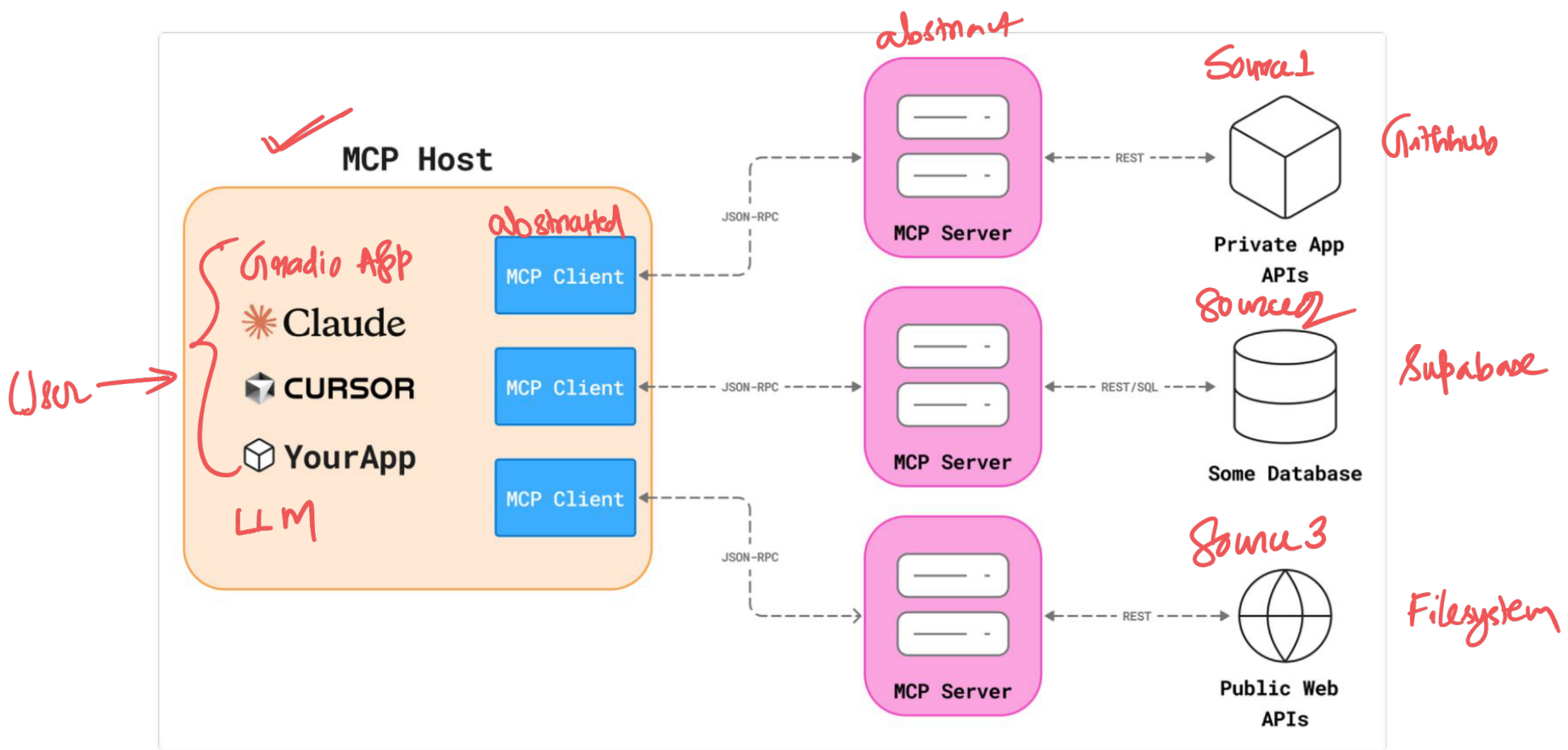
LLMs are indeed

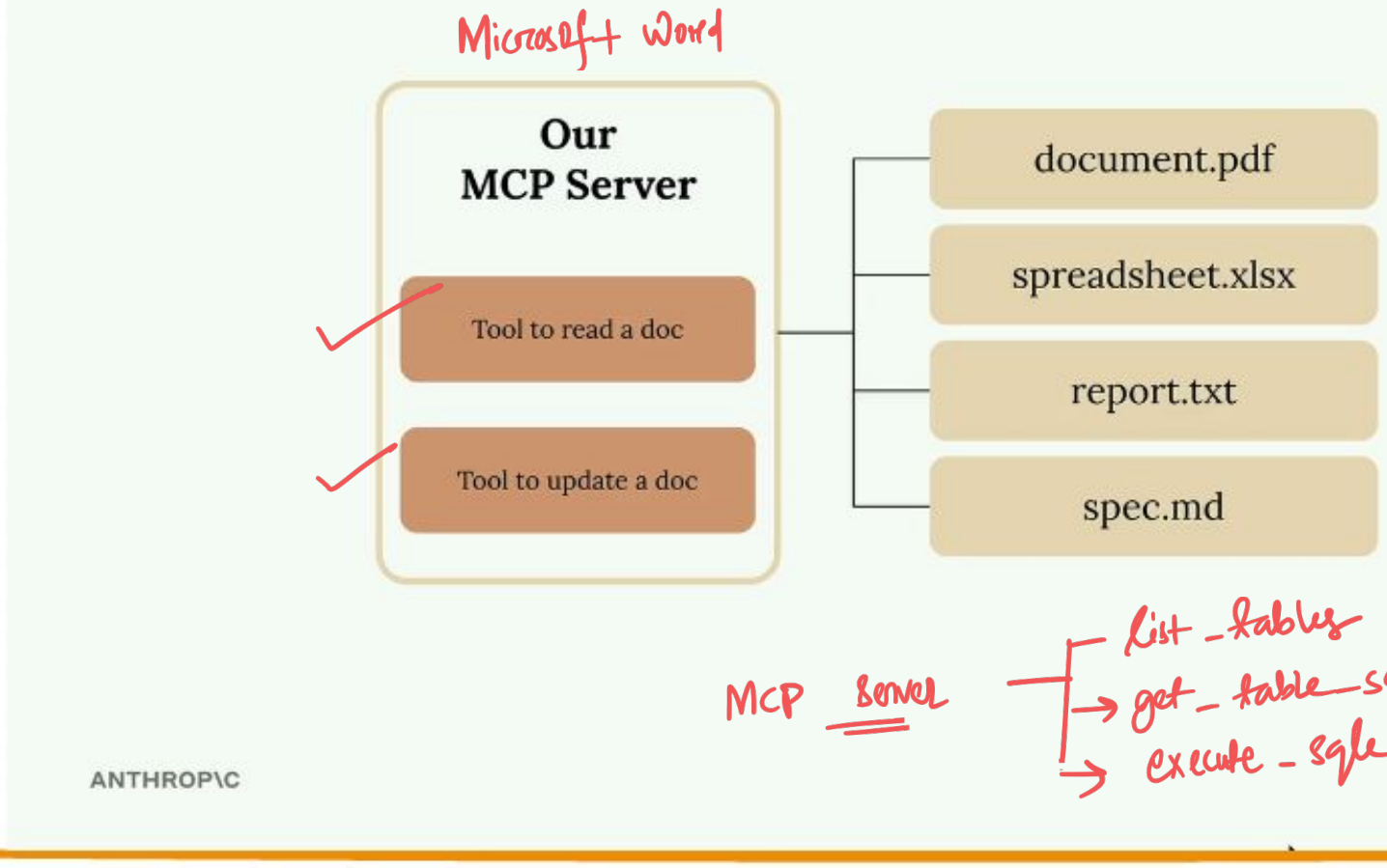
MCP Server →

1] Tools

2] Resources

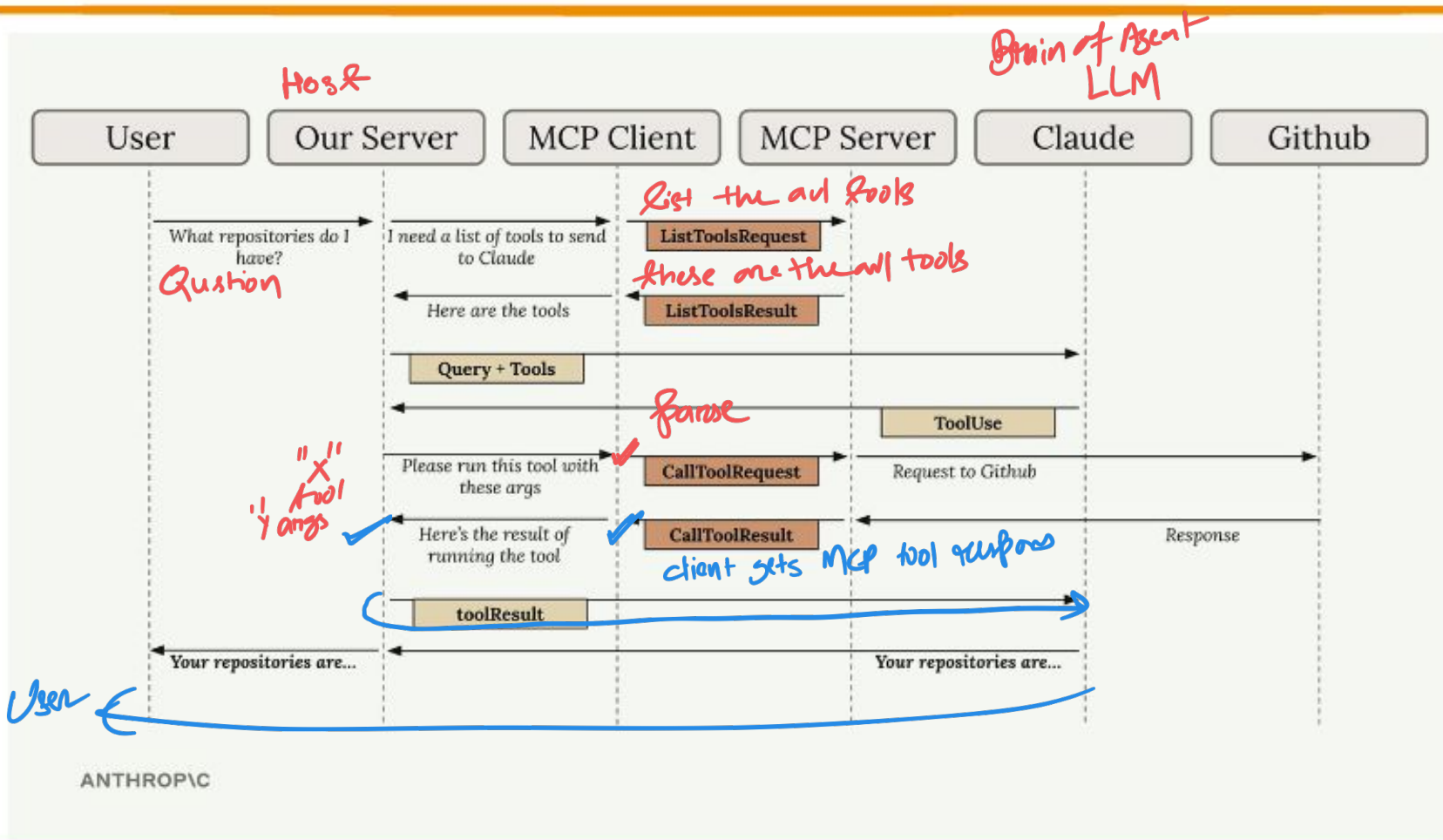
3] Specialized Prompts

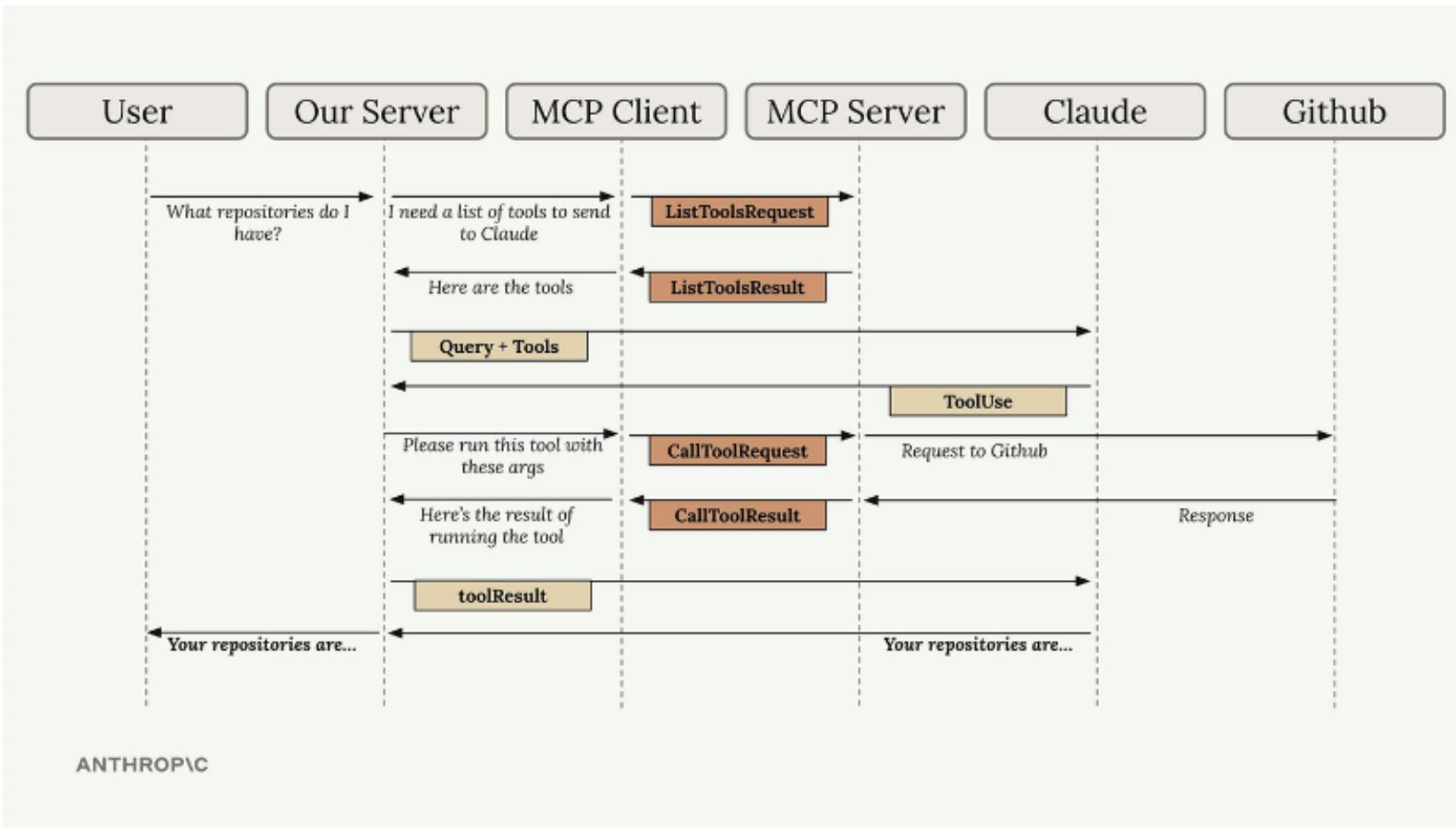

MCP Host
Claude
CURSOR
YourApp
MCP Client
MCP Client
MCP Client
JSON-RPC
JSON-RPC
JSON-RPC
MCP Server
MCP Server
MCP Server
REST
REST/SQL
REST
Private App APIs
Some Database
Public Web APIs



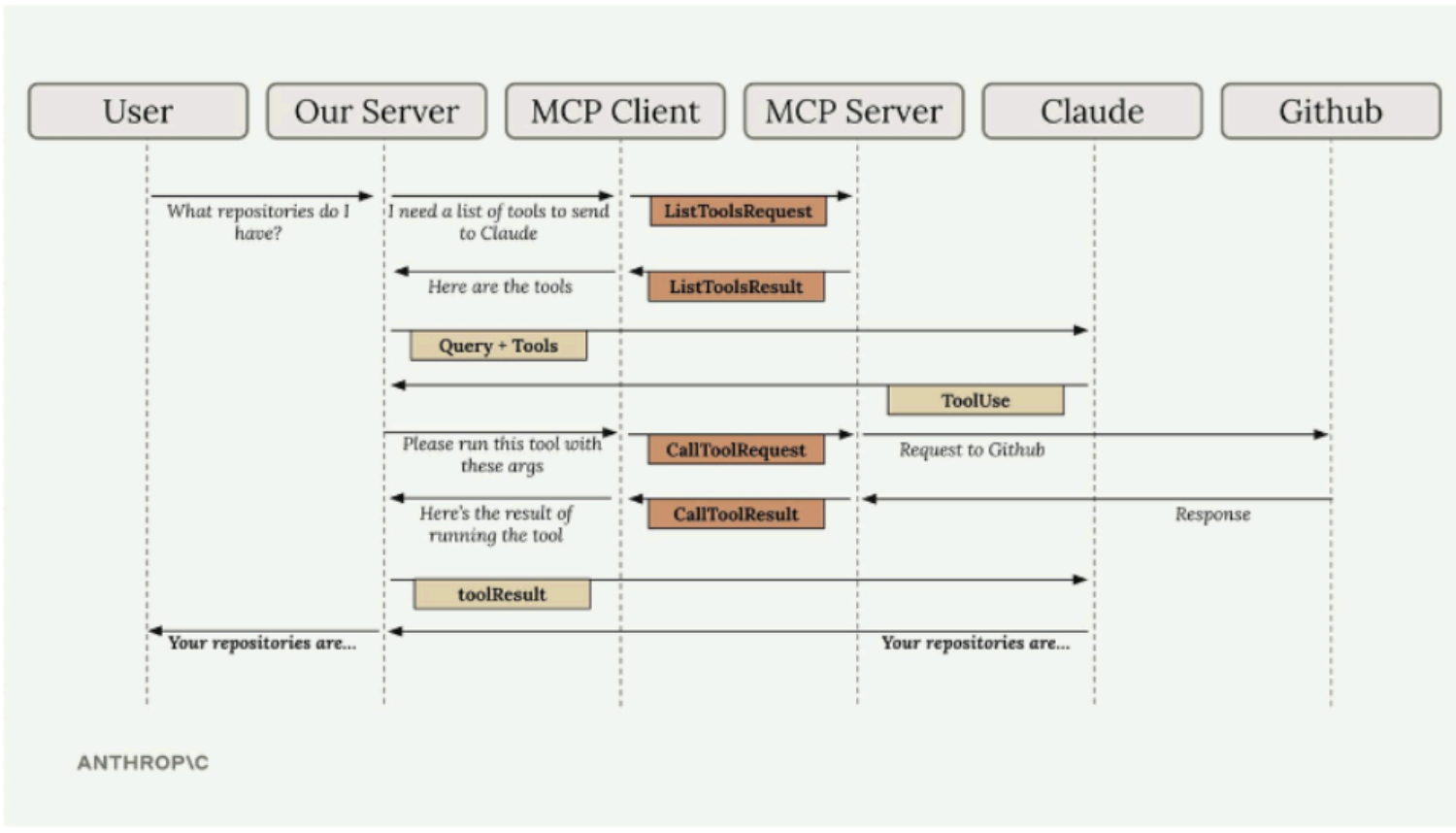
Microsoft Word

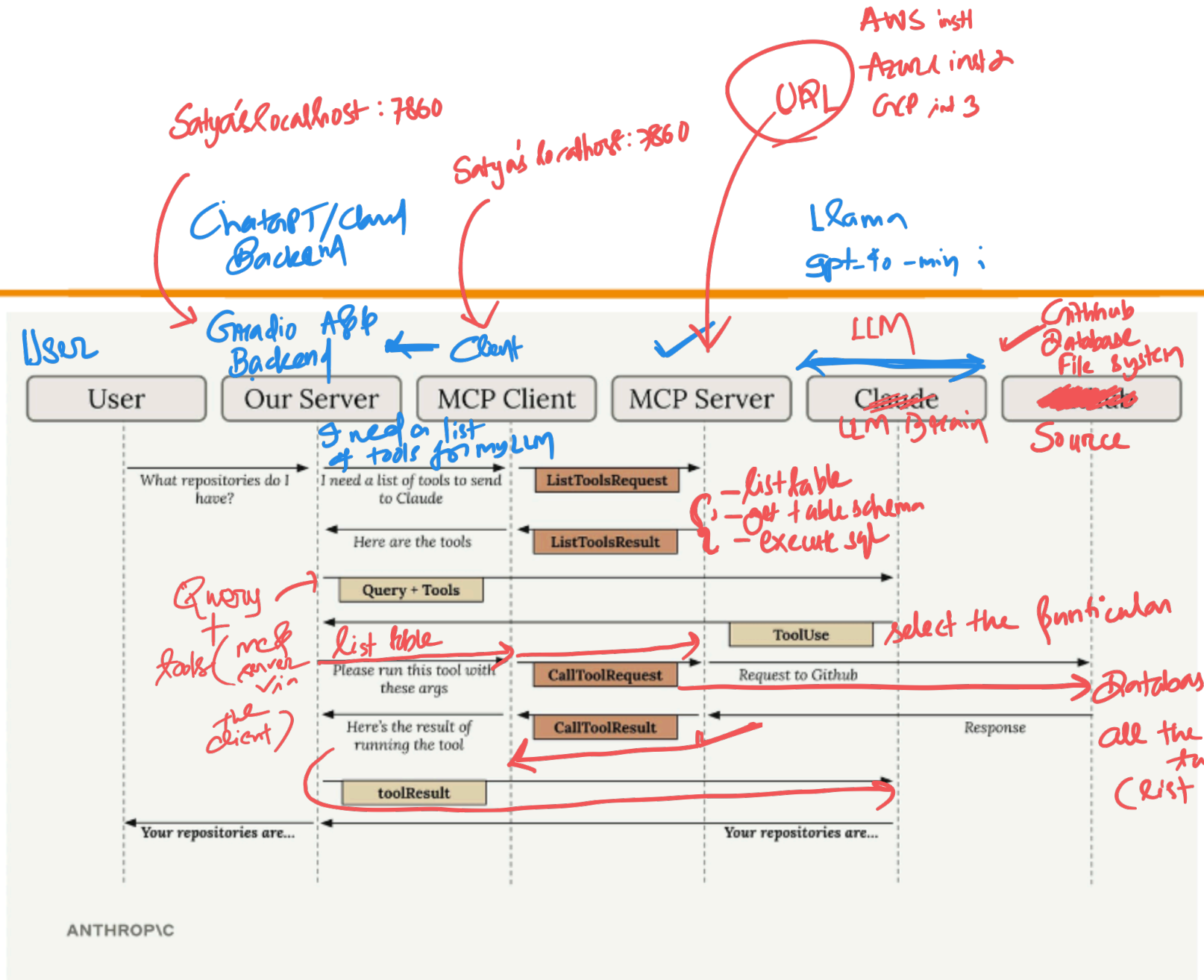
MCP Server → list_tables, get_table_schema, execute_sql


User
Our Server
MCP Client
MCP Server
Claude
Github
What repositories do I have?
I need a list of tools to send to Claude
ListToolsRequest
Here are the tools
ListToolsResult
Query + Tools
ToolUse
Please run this tool with these args
CallToolRequest
Request to Github
Here's the result of running the tool
CallToolResult
Response
toolResult
Your repositories are...
Your repositories are...
ANTHROP\C


User
Our Server
MCP Client
MCP Server
Claude
Github
What repositories do I have?
I need a list of tools to send to Claude
ListToolsRequest
Here are the tools
ListToolsResult
Query + Tools
ToolUse
Please run this tool with these args
CallToolRequest
Request to Github
Here's the result of running the tool
CallToolResult
Response
toolResult
Your repositories are...
Your repositories are...

ANTHROP\C


User
Our Server
MCP Client
MCP Server
Claude
Github
What repositories do I have?
I need a list of tools to send to Claude
ListToolsRequest
Here are the tools
ListToolsResult
Query + Tools
ToolUse
Please run this tool with these args
CallToolRequest
Request to Github
Here's the result of running the tool
CallToolResult
Response
toolResult
Your repositories are...
Your repositories are...
ANTHROP\C


User
Our Server
MCP Client
MCP Server
Claude
What repositories do I have?
I need a list of tools to send to Claude
ListToolsRequest
Here are the tools
ListToolsResult
Query + Tools
ToolUse
Please run this tool with these args
CallToolRequest
Request to Github
Here's the result of running the tool
CallToolResult
Response
toolResult
Your repositories are...
Your repositories are...
ANTHROP\C

10 mins

9:25


MCP
MCP
MCP
User
Host
Client
Server
Source
query
X

MCP Server: Code Flow Example

This walkthrough traces the implementation flow of an MCP Server that manages an in-memory document system, exposi
two resources, and one prompt.

Server Initialization

The Python MCP SDK reduces server creation to a single line. All primitives (tools, resources, prompts) are registered via d
the same server instance.

```python
from mcp.server.fastmcp import FastMCP

mcp = FastMCP("Document Server")
```

The backing data store is a simple in-memory dictionary:

Tool 1: Read Document

Flow: Decorator → typed parameters with `Field()` descriptions → validation → return content.

```
from pydantic import Field

@mcp.tool()
def read_doc_contents(doc_id: str = Field(description="ID of the document to
    if doc_id not in docs:
        raise ValueError(f"Document {doc_id} not found")
    return docs[doc_id]
```

mcp_server.py — AWS account = 172.0.0.1:8000

```
from mcp.server.fastmcp import FastMCP

mcp = FastMCP("Document Server")
```

database_mcp = FastMCP("Database Server")

Client Initialization

The Client wraps the MCP SDK's `ClientSession` in a class that manages connection lifecycle and resource cleanup.

```
from mcp import ClientSession

class MCPClient:
    def __init__(self):
        self.session = None

    async def __aenter__(self):
        await self.connect()
        return self

    async def __aexit__(self, *args):
        await self.cleanup()
```

Tool Discovery

Flow: Application needs tools for the LLM → Client queries Server → returns tool list.

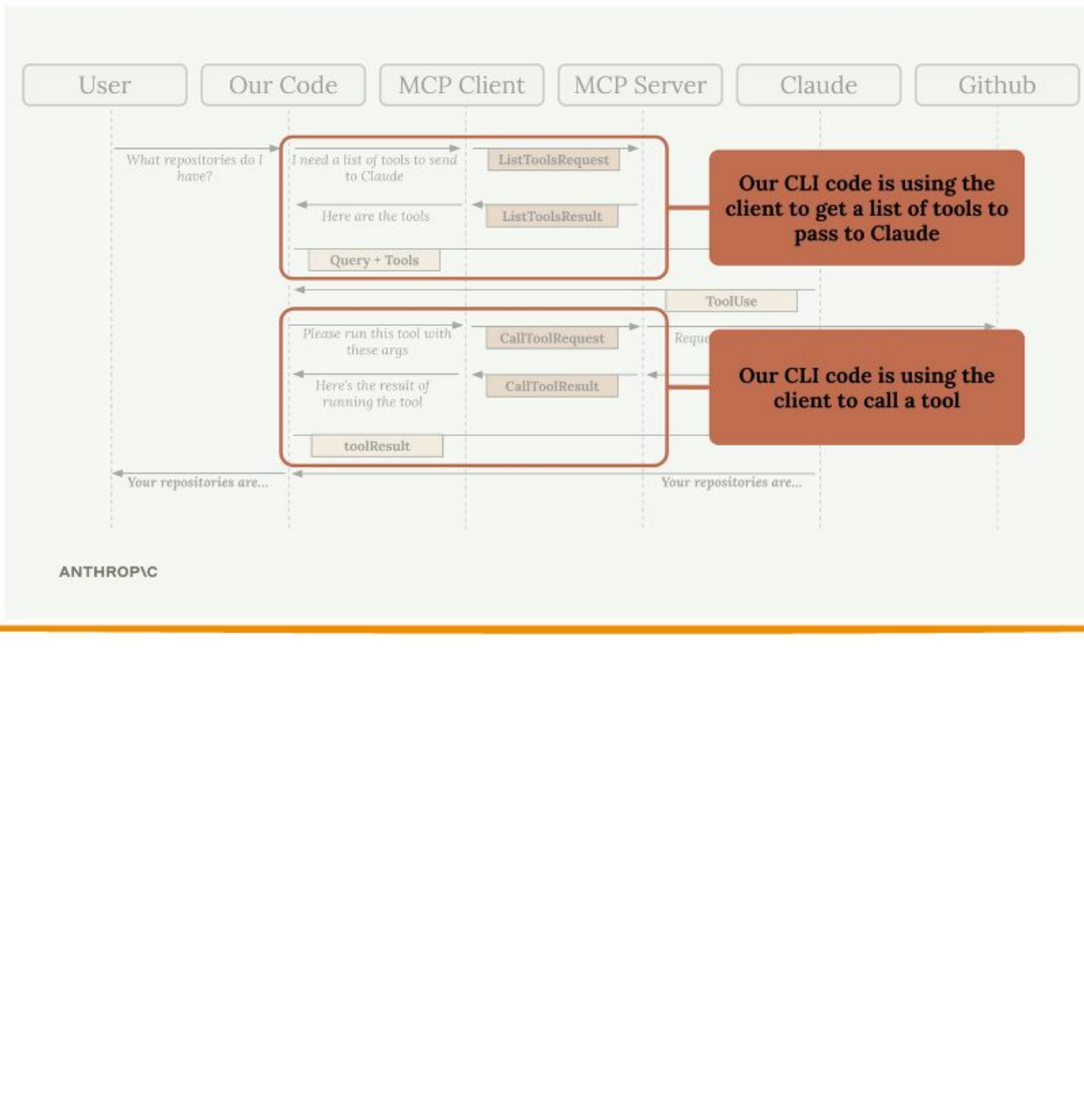
```python
async def list_tools(self):
    result = await self.session.list_tools()
    return result.tools
```

This sends a `tools/list` JSON-RPC request to the Server. The Server responds with its available tools and their schemas. The Client passes them through — it does not interpret or modify the schemas.

Tool Execution

Flow: LLM selects a tool and provides arguments → Client forwards to Server → Server executes → Client returns result.

```python
async def call_tool(self, tool_name: str, tool_input: dict):
    result = await self.session.call_tool(tool_name, tool_input)
    return result
```


User
Our Code
MCP Client
MCP Server
Claude
Github
What repositories do I have?
I need a list of tools to send to Claude
ListToolsRequest
Here are the tools
ListToolsResult
Query + Tools
Our CLI code is using the client to get a list of tools to pass to Claude
ToolUse
Please run this tool with these args
CallToolRequest
Here's the result of running the tool
CallToolResult
toolResult
Our CLI code is using the client to call a tool
Your repositories are...
Your repositories are...
ANTHROP\C

```python
@mcp.resource(
    "docs://documents", # URI
    mime_type="application/json"
)
def list_docs():
    # Return a list of document names
```

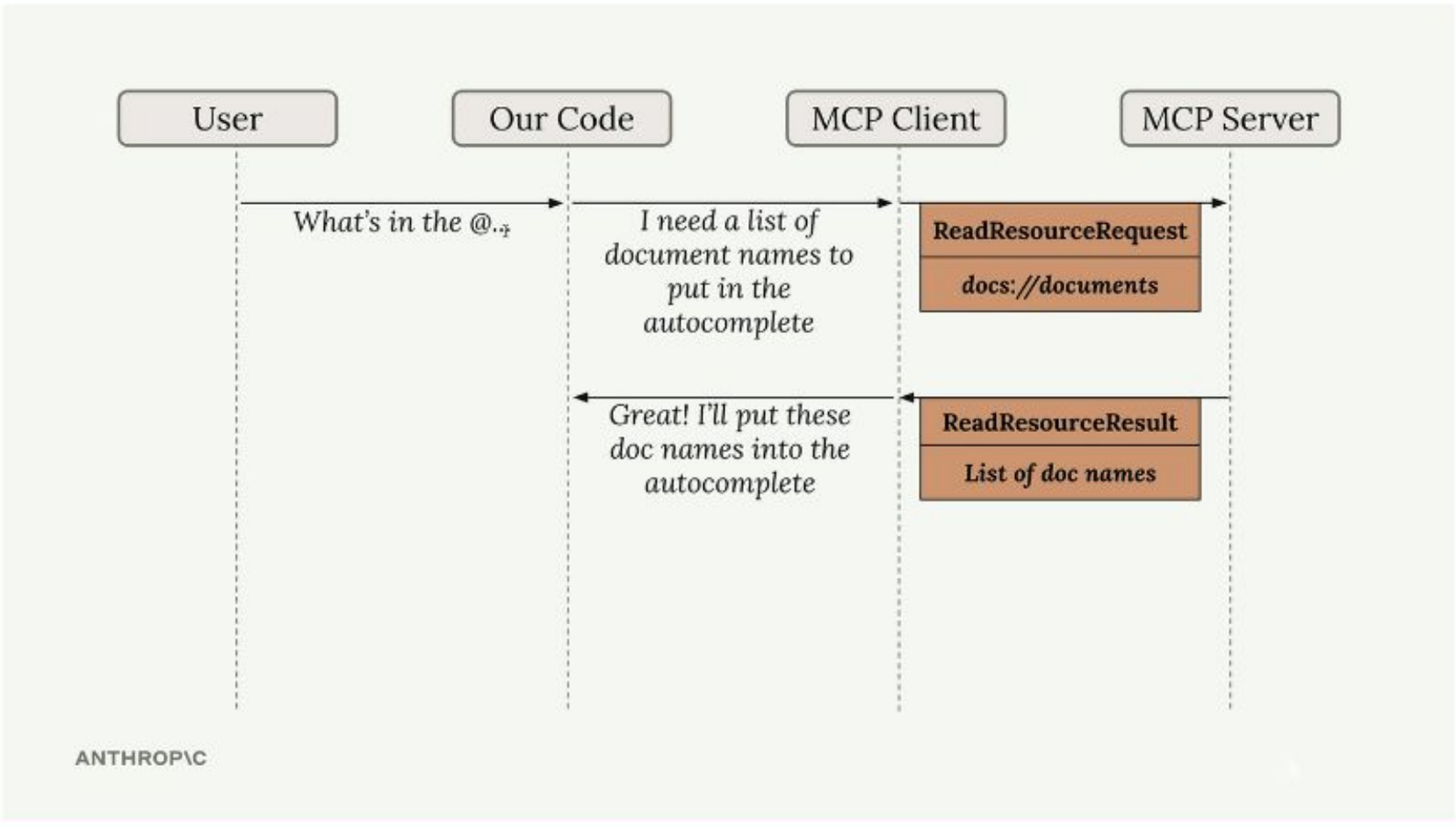
Direct Resource

URI doesn't contain any params.

```python
@mcp.resource(
    "docs://documents/{doc_id}", # URI
    mime_type="text/plain"
)
def fetch_doc(doc_id: str):
    # Return the contents of a doc
```

Templated Resource

URI contains one or more params. The Python SDK parses these and passes them as args to your function.


User
Our Code
MCP Client
MCP Server
What's in the @...
I need a list of document names to put in the autocomplete
ReadResourceRequest
docs://documents
Great! I'll put these doc names into the autocomplete
ReadResourceResult
List of doc names

Client Interface

The Client exposes a single function for reading resources:

```
import json
from pydantic import AnyUrl

async def read_resource(self, uri: str):
    result = await self.session.read_resource(AnyUrl(uri))
    resource = result.contents[0]

    if resource.mime_type == "application/json":
        return json.loads(resource.text)
    return resource.text
```

If we left this process up to a user, here's what they'd write:

```
Convert report.pdf to markdown
```

Yes, it'd work, but the user might get a better result with some strong prompt engineering

ANTHROP\C

User might have more luck if they use our thoroughly-eval'd prompt instead!

```
You are a document conversion specialist tasked with rewriting documents in Markdown
format. Your goal is to take the content of a given document and convert it into
well-structured Markdown, preserving the original meaning and enhancing readability.
Here is the identifier of the document you need to convert:
<document_id>
{{doc_id}}
</document_id>
Instructions:
1. Retrieve the content of the document associated with the given document_id.
2. Analyze the structure and content of the document.
3. Convert the document to Markdown format, following these guidelines:
- Use appropriate header levels (# for main titles, ## for subtitles, etc.)
- Properly format lists (both ordered and unordered)
- Use emphasis (*italic* or **bold**) where appropriate
- Add links and images using Markdown syntax if present in the original document
- Preserve any special formatting or structure that's important to the document's
meaning
Before providing the final Markdown output, in <document_analysis> tags:
- Identify the main sections and subsections of the document
- Count the number of sections and subsections to ensure proper nesting of headers
This will help ensure a thorough and well-organized conversion.
After your analysis, present the converted document in Markdown format. Use ```
markers to denote the beginning and end of the Markdown content.
Example output structure:
<document_analysis>
[Your analysis of the document structure and conversion plan]
</document_analysis>
```markdown
# Document Title
## Section 1
Content of section 1...
## Section 2
Content of section 2...
- List item 1
- List item 2
[Link text](https://example.com)
![Image description](image-url.jpg)
...
```
Please proceed with your analysis and conversion of the document.
```

Client Interface

The Client exposes two functions for working with prompts:

Discovering Available Prompts

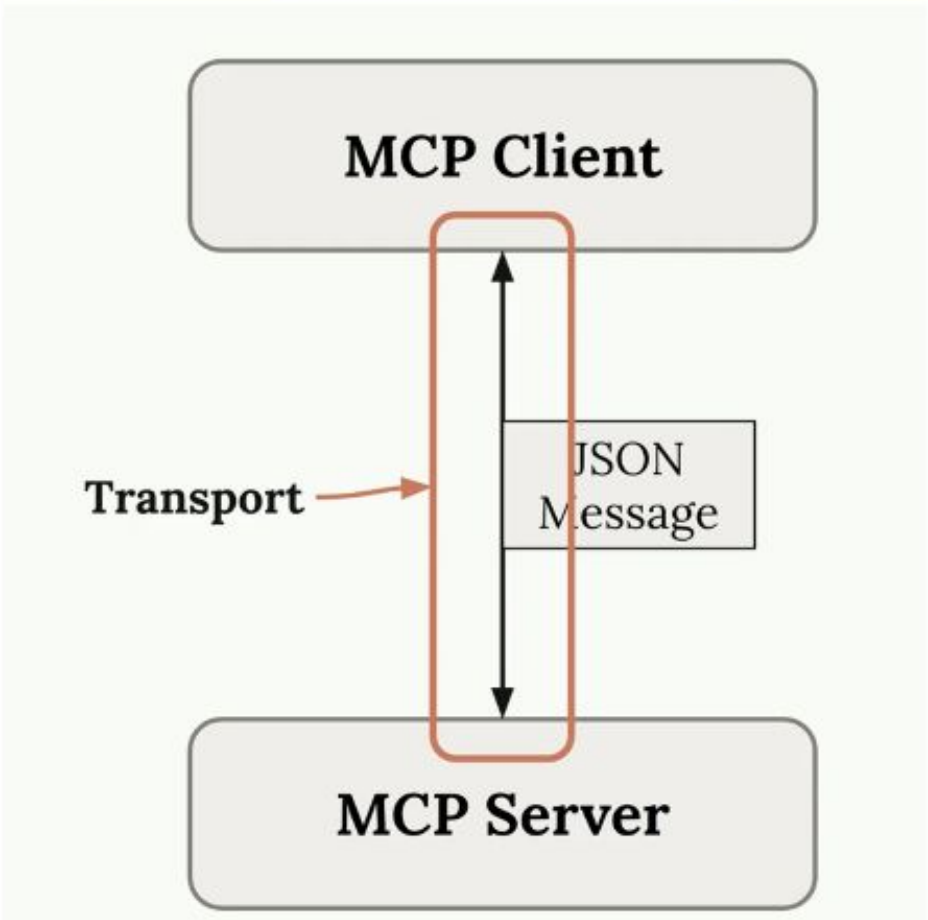
```python
async def list_prompts(self):
    result = await self.session.list_prompts()
    return result.prompts
```

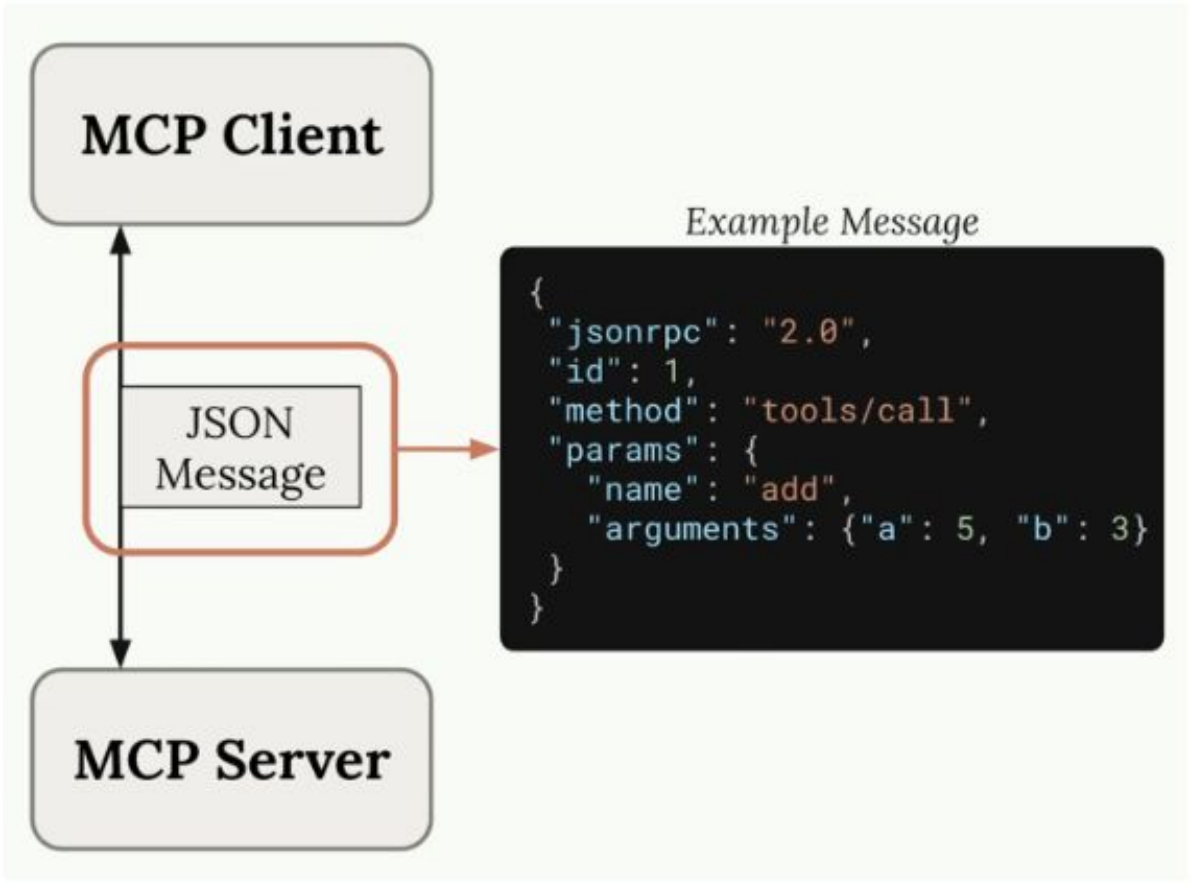
Sends a `list_prompts` request to the Server. Returns metadata (name, description, expected arguments) for every prompt the Server offers. The application uses this to render available options to the user — slash commands in a CLI, buttons in a UI.

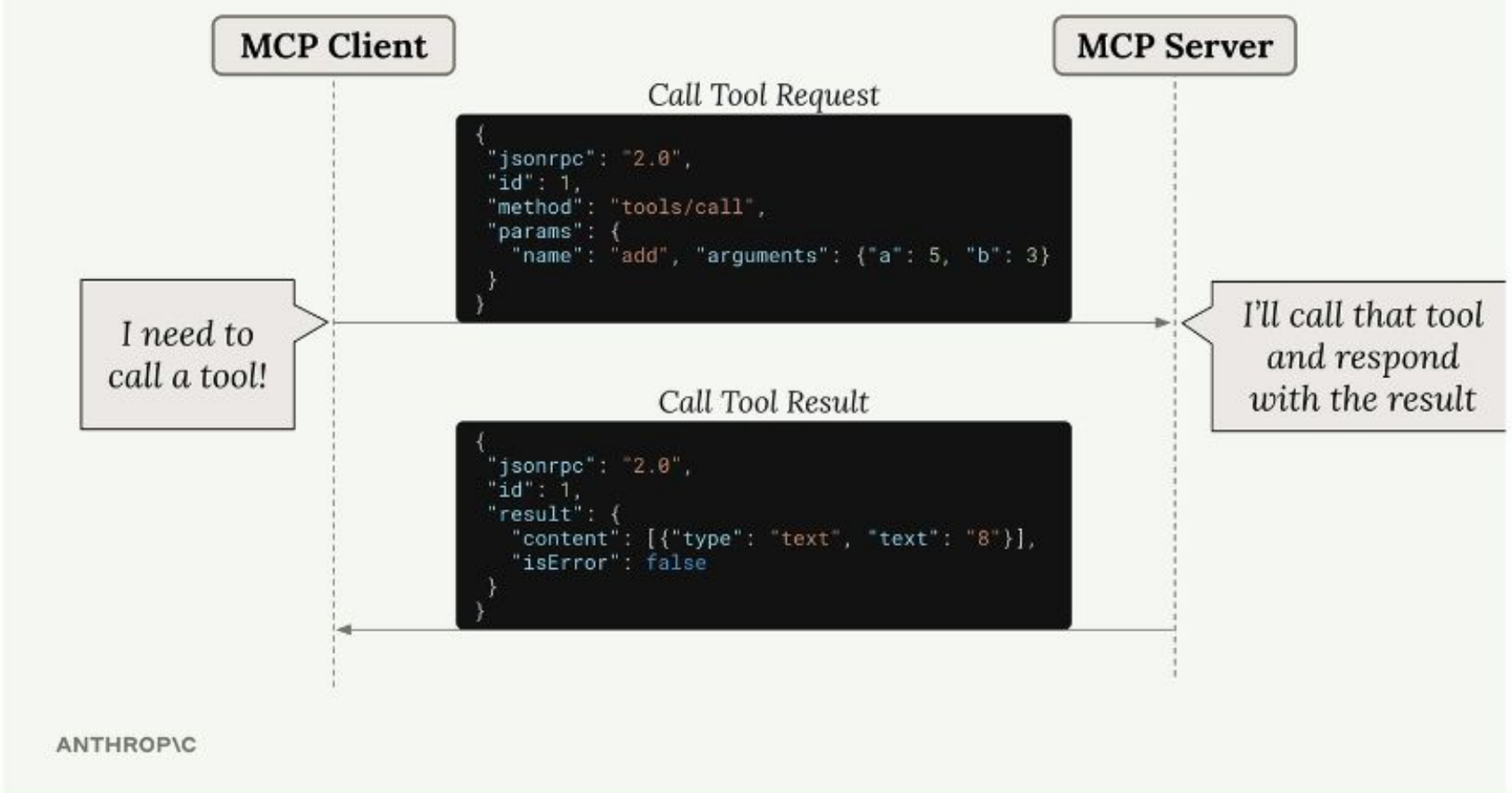
Retrieving a Specific Prompt

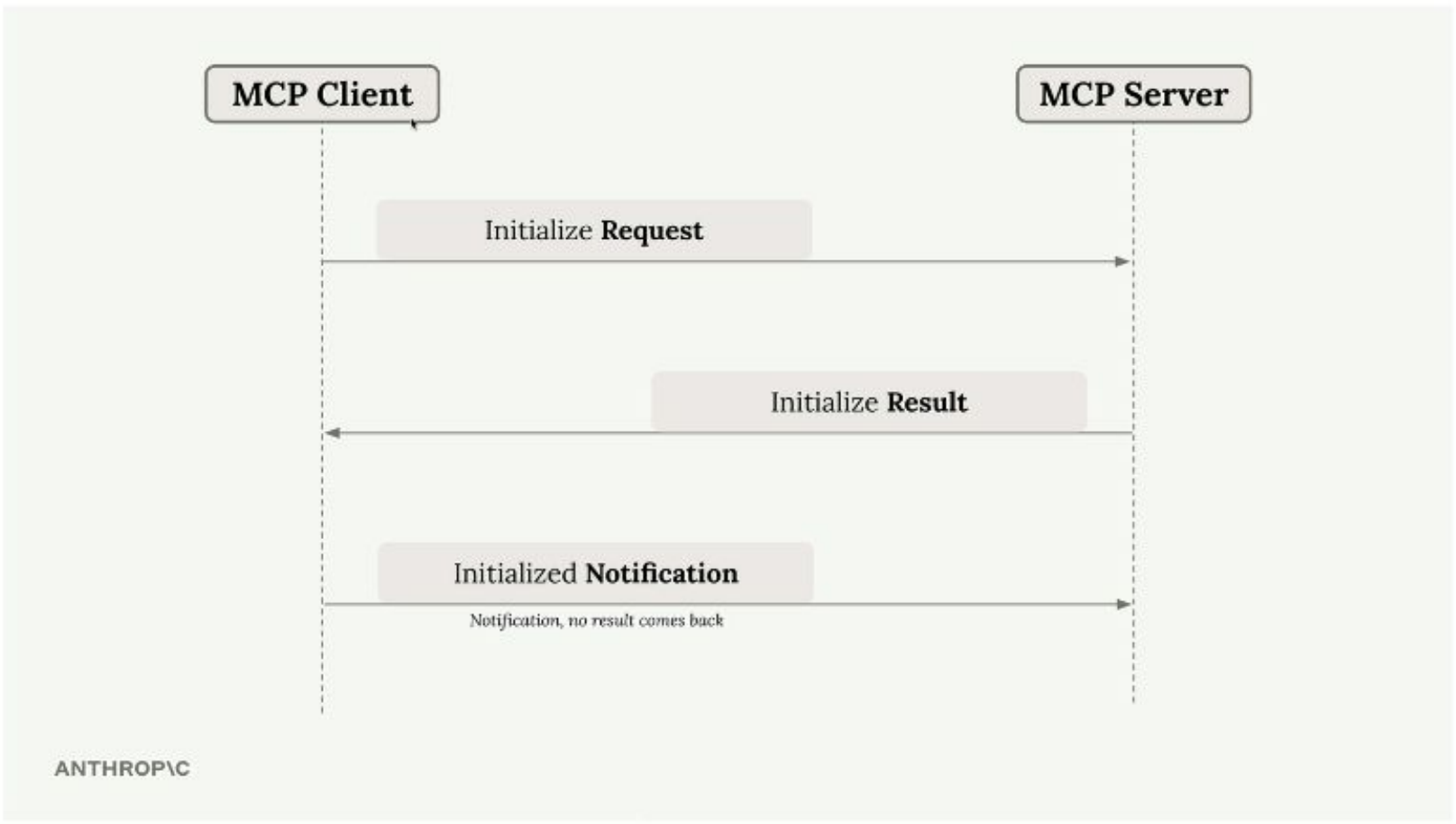
```python
async def get_prompt(self, prompt_name: str, arguments: dict):
    result = await self.session.get_prompt(prompt_name, arguments)
    return result.messages
```

Sends a `get_prompt` request with the prompt name and a dictionary of arguments. The Server interpolates the arguments into its template and returns a **messages array** — a ready-to-send conversation sequence for the LLM.


MCP Client
Transport
JSON
Message
MCP Server


MCP Client
JSON
Message
MCP Server
Example Message
{
"jsonrpc": "2.0",
"id": 1,
"method": "tools/call",
"params": {
"name": "add",
"arguments": {"a": 5, "b": 3}
}
}


MCP Client
MCP Server
Call Tool Request
{
"jsonrpc": "2.0",
"id": 1,
"method": "tools/call",
"params": {
"name": "add", "arguments": {"a": 5, "b": 3}
}
}
I need to call a tool!
I'll call that tool and respond with the result
Call Tool Result
{
"jsonrpc": "2.0",
"id": 1,
"result": {
"content": [{"type": "text", "text": "8"}],
"isError": false
}
}
ANTHROP\C


MCP Client
MCP Server
Initialize **Request**
Initialize **Result**
Initialized **Notification**
Notification, no result comes back
ANTHROP\C

stdin → program's standard in stream

stdout → program's standard out stream

Two ways of comm betw MCP client/server

→ Stdin / Stdout

→ HTTP Streamable transport (SSE)

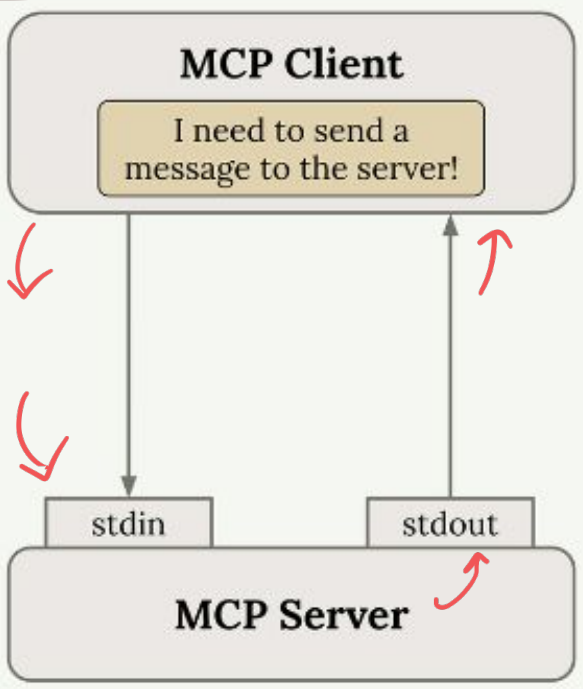
How can we implement each of these with stdio?

Initial request from **Client** → Server

Response from **Server** → Client

Initial request from **Server** → Client

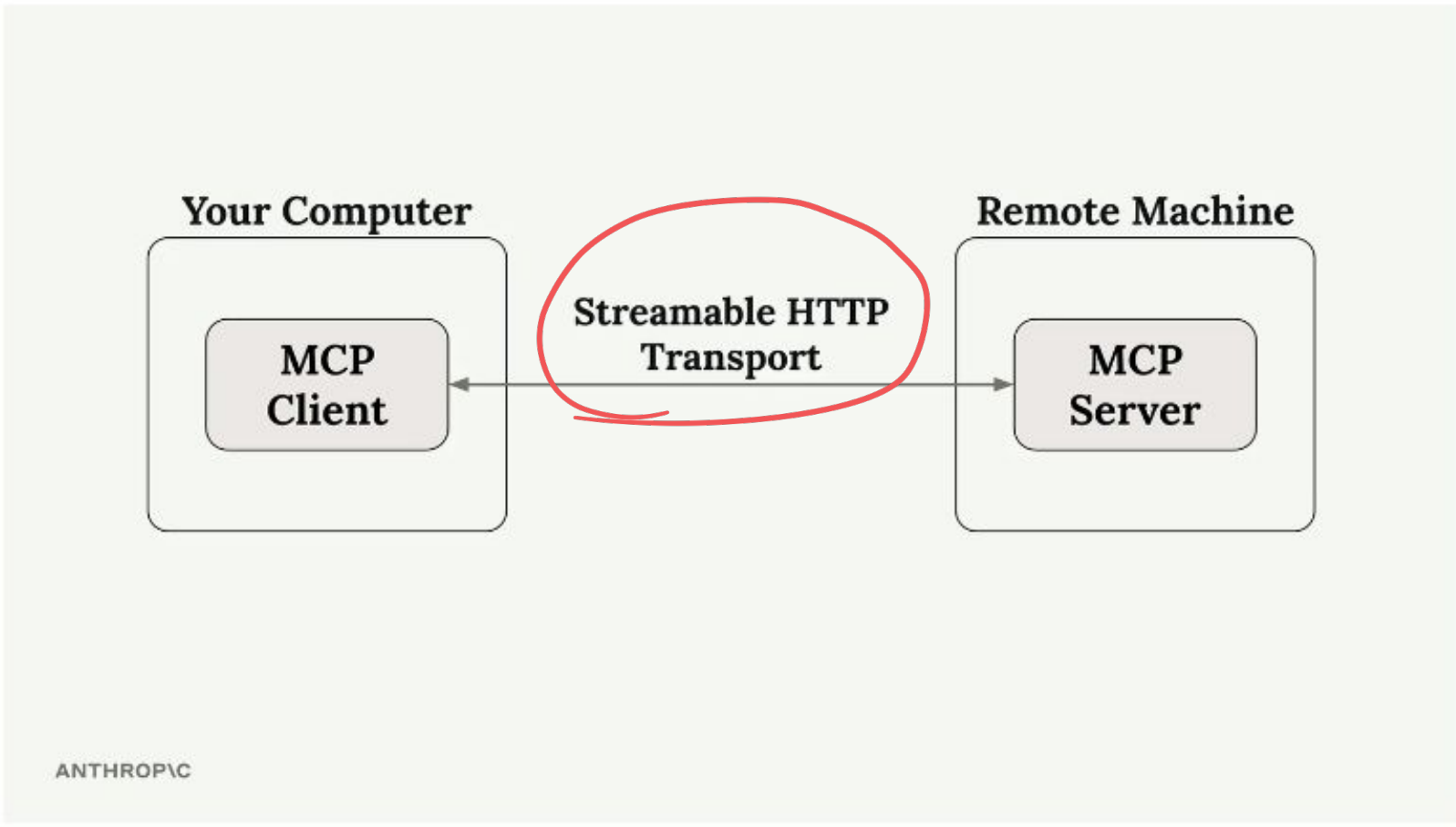
Response from **Client** → Server



Can I have a AWS instance MCP server on a diff machine and a MCP client on a different machine?

↓

Azure machine


Your Computer
MCP Client
Streamable HTTP Transport
Remote Machine
MCP Server
ANTHROP\C

Server Implementation

Defining a Prompt

Flow: Decorator with name and description → function receives arguments → returns a message list.

```
@mcp.prompt(name="format", description="Rewrites a document in markdown format")
def format_document(doc_id: str) -> list:
    return [
        {
            "role": "user",
            "content": f"Read the document '{doc_id}' using the read_doc_contents tool,
                       f"rewrite its contents in clean markdown format, "
                       f"then save the result using the edit_document tool."
        }
    ]
```

LLM — How many customers pay in "EURO"

get SQL — where currency = "EURO" "EUR"

"EUR", "DOLLAR", "YEN"

get unique values of currency → "currency"

"get unique values of a column"